

BioSpecInfo

Matrice di tracciabilità — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v186** · commit [e486483](#)

Generato il 2026-10-03

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Matrice di tracciabilità — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Versione descritta	bsi-v186
Scopo	Collegare ogni requisito dichiarato all'implementazione che lo realizza e al banco di prova che lo verifica.

Come leggere questa matrice. Ogni riga è una catena chiusa: un requisito, il punto del codice che lo realizza, e il banco che ne verifica il funzionamento. Una riga senza banco è un requisito **non verificato** e viene dichiarata come tale, non nascosta: l'utilità di una matrice sta esattamente nel rendere visibili le lacune.

Gli esiti dei banchi citati sono riportati per esteso, con l'uscita integrale, in [evidence/RAPPORTO-VERIFICA.md](#), generato automaticamente.

1. Requisiti scientifici

ID	Requisito	Implementazione	Banco di verifica
SCI-01	I dati strutturali dei farmaci devono essere coerenti con il peso molecolare dichiarato	FARM_DATA in <code>index.html</code>	<code>tools/verifica-farmaci.js</code> — confronto RDKit ↔ letteratura su 233 voci
SCI-02	I gruppi funzionali devono essere riconosciuti sulla struttura, non sul testo dello SMILES	<code>bsi-spettri.js</code> §1, pattern SMARTS via RDKit	<code>test_spettri</code> — 8 molecole di riferimento, riconoscimento additivo
SCI-03	Una molecola con più gruppi funzionali deve mostrarli tutti	<code>bandeDaGruppi()</code> , costruzione additiva	<code>test_spettri</code> §1 — acido acetilsalicilico: estere e acido
SCI-04	Gli spettri devono rispettare le convenzioni della tecnica	<code>svgIR()</code> , <code>makeNMRsvg()</code> , <code>drawUVSpectrum()</code> , <code>drawMSSpectrum()</code>	<code>test_assi</code> (SVG), <code>test_assi_canvas</code> (canvas)
SCI-05	Uno spettro calcolato deve essere riproducibile	rimozione di ogni <code>Math.random()</code> dalle tracce	<code>test_assi</code> §3, <code>test_spettri</code> §8 — confronto byte a byte
SCI-06	L'altezza dei picchi ¹ H deve rappresentare l'integrazione o non essere asserita	<code>makeNMRsvg()</code> ; <code>_nmrSvg()</code> disegna fasce a altezza uniforme	<code>test_assi</code> §4
SCI-07	La coniugazione deve agire sul singolo carbonile	SMARTS <code>acidoConiug</code> , <code>estereConiug</code> , <code>chetoneConiug</code> , <code>aldeideConiug</code>	<code>test_spettri</code> §1 — aspirina 1760/1690; acetofenone 1690

ID	Requisito	Implementazione	Banco di verifica
SCI-08	Una costante fisica ambigua non deve essere risolta arbitrariamente	<code>costante_fisica</code> in <code>bsi-ai-hub.js</code> — raccoglie tutte le candidate	<code>test_costanti</code> — 45 controlli, compresi i rifiuti per ambiguità
SCI-09	I limiti dei predittori devono essere dichiarati all'utente	intestazione dei grafici; <code>docs/06 §3.5</code>	<code>verifica_guida</code> — coerenza fra promesse e codice
SCI-10	La simmetria molecolare deve determinare correttamente le regole di selezione	sezione <code>ssimm</code> in <code>index.html</code>	<code>test_simmetria</code> — 26 controlli: modi normali, centro di inversione, esclusione mutua
SCI-12	Un cromatogramma deve discendere dai parametri, non essere disegnato	sezione <code>scroma</code> : $\sigma = t_{R}/\sqrt{N}$, R_s da distanza e larghezze	<code>audit_stabilita</code> (apertura e disegno), ispezione dei valori contro le definizioni
SCI-13	Una retta di taratura deve dichiarare l'incertezza dei suoi coefficienti e riconoscere i due modi in cui inganna	sezione <code>staratura</code> : minimi quadrati ordinari, test F retta↔parabola, residuo cancellato	Verificato sui tre insiemi di esempio inclusi: lineare ($F = 0,3$), non lineare ($F = 186,6 > 7,71$), con punto anomalo (1 rilevato)
SCI-14	Un modello QSAR deve essere valutato su scheletri mai visti, non su analoghi dello stesso insieme	<code>bsi-cheminfo.js §9</code> — <code>divisionePerScaffold()</code> accanto a <code>divisioneCasuale()</code>	<code>test_cheminfo</code> — la divisione per scheletro non lascia nessuno scheletro in comune fra addestramento e prova
SCI-15	Un punteggio di modello deve essere confrontato con il caso, non presentato da solo	<code>bsi-cheminfo.js §9</code> — riaddestramento su etichette rimescolate, otto ripetizioni	<code>test_cheminfo</code> — verificato nei due versi : su un segnale apprendibile $R^2 0,394$ batte tutti gli otto sosia (margine 0,287); su etichette casuali $R^2 -0,199$ non li batte
SCI-16	Due scritture della stessa molecola non devono contarsi due volte né finire da parti opposte della divisione	<code>bsi-cheminfo.js §1</code> — deduplicazione sullo SMILES canonico dopo il frammento maggiore	<code>test_cheminfo</code> — <code>OC(=O)C</code> e <code>CC(=O)O</code> collassano in una voce; i sali perdono il controione
SCI-17	La similarità fra molecole deve essere calcolata, non stimata	<code>bsi-cheminfo.js §3</code> — Tanimoto e Dice su impronte Morgan, RDKit, pattern, MACCS	<code>test_cheminfo</code> — confronto con valori calcolabili a mano, compresa la convenzione $0/0 = 1$
SCI-18	Una validazione su una divisione sola, su un insieme piccolo, è un numero rumoroso: va accompagnata dalla sua dispersione	<code>bsi-cheminfo.js §9-bis</code> — <code>validazioneIncrociata()</code> su pieghe raggruppate per scheletro	<code>test_cheminfo</code> — le pieghe coprono ogni molecola una volta sola, nessuno scheletro sta in due pieghe , e con lo stesso seme le pieghe sono le stesse
SCI-19	Un risultato che cambia a ogni esecuzione non è verificabile da nessuno	generatore <code>mulberry32</code> seminato per divisioni, rimescolamenti e pieghe; il seme è scritto nel rapporto	<code>test_cheminfo</code> — due chiamate con lo stesso seme danno pieghe identiche; il rapporto di metodo dichiara il seme

ID	Requisito	Implementazione	Banco di verifica
SCI-20	I risultati devono poter uscire dallo strumento, e il metodo deve uscire con loro	<code>bsi-cheminfo.js</code> §9-ter — <code>esportaCsv()</code> e <code>rapportoMetodo()</code> , entrambi formati in memoria nel browser	<code>test_cheminfo</code> — colonne coerenti con l'intestazione, nome con virgola citato, residuo calcolato, cella vuota per l'attività mancante; il rapporto contiene seme, fingerprint, verdetto, limiti
SCI-21	Un numero presentato come similarità strutturale deve venire da un fingerprint, e una sola implementazione deve calcolarlo	<code>rdkit_lab.html</code> carica <code>bsi-cheminfo.js</code> e ne usa <code>tanimoto()</code> e <code>fingerprint()</code> ; la similarità a otto bit è stata rimossa	<code>test_cheminfo</code> — la pagina e il motore danno lo stesso Tanimoto su tutte e 28 le coppie , scarto esattamente 0 ; le funzioni della similarità finta non esistono più; ogni struttura di riferimento è valida e ha la massa monoisotopica del farmaco che dichiara
SCI-22	Le conversioni fra linguaggi molecolari devono essere verificate contro una fonte esterna o contro il calcolo a mano, e le coordinate scritte in un file devono essere vere e dichiarate	<code>bsi-molingue.js</code> — diciassette uscite , da SMILES a CML	<code>test_lingue</code> — 7 chiavi InChI di letteratura , giro completo SMILES → molfile → SMILES, 26 nomi a mano, 7 rifiuti pretesi ; composizione in massa contro il calcolo a mano (alanina C 40,44%, aspirina C 60,00%); XYZ e PDB con coordinate non nulle e dichiarate 2D
SCI-23	Angoli, lunghezze e diedri si misurano sulle coordinate, e su coordinate piatte non si mostrano	prodotto scalare per gli angoli, prodotto vettoriale per i diedri	<code>test_mol3d</code> — tetraedro 109,4712° , acqua 104,47°, CO ₂ 180°, BF ₃ 120°, ammoniaca 106,13°; diedri 0°, 60°, 90°, 180°; C-C dell'etanolo 1,509 Å; e il rifiuto provato nei due versi

2. Requisiti dell'agente AI

ID	Requisito	Implementazione	Banco di verifica
AI-01	Un modello ritirato dal fornitore non deve bloccare l'assistente	<code>modelloSuggestitoDaErrore()</code> , lista nera <code>bsi_modelli_ko</code>	<code>test_404</code> — 36 controlli
AI-02	Un fornitore irraggiungibile dal browser deve essere ricordato e retrocesso	<code>bsi_prov_ko</code> , TTL 24 h, <code>providerUtilizzabili()</code>	<code>test_ko</code> (35), <code>browser_ko</code> (37)
AI-03	Un sovraccarico temporaneo deve produrre un nuovo tentativo, non un errore	<code>erroreTemporaneo()</code> , <code>TENTATIVI_TEMPORANEO</code>	<code>test_503</code> — 40 controlli
AI-04	Un'attesa imposta superiore a 10 s deve far cambiare fornitore se ve n'è uno pronto	<code>ATTESA_TROPPO_LUNGA_MS</code>	<code>test_attesalunga</code> — 14 controlli

ID	Requisito	Implementazione	Banco di verifica
AI-05	Il tetto di token del fornitore deve essere appreso dall'errore, non codificato	<code>tettoDaErrore()</code> , <code>MARGINE_TETTO</code>	<code>test_tetto</code> — 32 controlli
AI-06	Le firme di ragionamento opache devono essere restituite al fornitore	cattura e replay in <code>appendAgentTurn()</code>	<code>test_firma</code> — 31 controlli
AI-07	Due invii sovrapposti non devono corrompere la cronologia	<code>_turnoInCorso</code> in <code>send()</code>	<code>test_doppioinvio</code> — 7 controlli
AI-08	L'utente deve poter misurare quali fornitori raggiunge davvero	<code>provaTuttiIFornitori()</code>	<code>browser_prova</code> — 29 controlli
AI-09	I fornitori a pagamento non devono essere proposti come gratuiti	separazione per <code>PROVIDERS[id].free</code>	<code>browser_prova</code> §D2
AI-10	Deve esistere una via documentata per eliminare CORS e limiti	<code>proxy/spectra-proxy.js</code> + guida in-app	<code>test_guidaproxy</code> (12), <code>browser_proxy</code> (15), <code>browser_proxyui</code> (17)

3. Requisiti di stabilità

ID	Requisito	Implementazione	Banco di verifica
STA-01	Una sessione lunga non deve accumulare nodi DOM né timer	gestione del ciclo di vita delle sezioni	<code>audit_stabilita</code> §1 — 91 sezioni × 5 giri
STA-02	L'esaurimento di <code>localStorage</code> non deve rendere inutilizzabile l'app	scritture protette; avviso persistente nel File Manager	<code>audit_quota</code> (10), <code>test_filemanager</code> (15)
STA-03	Nessuna promessa rifiutata deve restare non gestita	<code>.catch()</code> sistematici	<code>audit_promesse</code> — 22 controlli, rete sana e rete morta
STA-04	Dati salvati corrotti non devono impedire l'avvio	<code>loadJSON()</code> con ripiego	<code>audit_stabilita</code> §4
STA-05	Una rete lenta non deve essere trattata come una rete assente	<code>ATTESA_RETE_MS = 3500</code> in <code>sw.js</code>	<code>test_sw</code> §3 e §3-bis — misurato 3 507 ms
STA-06	Un aggiornamento non deve distruggere il lavoro in corso	messaggio <code>BSI_SW_UPDATED</code> , decisione lato pagina	<code>test_sw</code> §4
STA-07	Sfogliare molecole non deve esaurire i contesti WebGL	riuso del visore 3D nei 5 punti di creazione	<code>test_visore3d</code> — da 7 costruzioni a 1
STA-08	L'archivio IndexedDB non disponibile deve essere dichiarato, non fatale	<code>dbStore()</code> su promessa; avviso persistente	<code>test_filemanager</code> §2

4. Requisiti di interfaccia e accessibilità

ID	Requisito	Implementazione	Banco di verifica
UI-01	Ogni pagina deve aprirsi senza errori JavaScript	—	<code>audit_stabilita</code> §5 — 14 pagine
UI-02	I grafici devono essere nitidi su schermi ad alta densità	<code>bsiNitido()</code> sui contesti canvas	<code>audit_grafici</code> — 40 canvas
UI-03	L'app deve funzionare a 390 px di larghezza	griglie <code>auto-fit</code> , nessuna colonna fissa	<code>audit_mobile</code> — <code>scrollWidth</code> del documento su tutte le 91 sezioni a 390 px; <code>audit_stabilita</code> per gli errori in viewport telefono
UI-04	L'utente deve poter sapere quale versione sta usando e forzare l'aggiornamento	voce «Aggiornamenti» nel pannello ✨	<code>test_aggiorna</code> — 9 controlli
UI-05	La cancellazione dei dati deve essere selettiva e reversibile nelle scelte	<code>bsiCancellaDati()</code> per gruppi	<code>browser_reset</code> — 24 controlli
UI-08	Anche le superfici che compaiono solo dopo un'azione devono rispettare il contrasto WCAG AA	i sei pannelli della sezione Chemioinformatica	<code>test_cheminfo</code> — l'analisi viene eseguita, i pannelli aperti a turno, 460 elementi di testo misurati, 0 difetti , con le formule WCAG riscritte dentro il banco
UI-09	Aprire una sezione non deve bloccare la pagina	primo <code>render()</code> di 3Dmol fuori dal click; le 296 figure di sintesi disegnate a fette da 8 ms	<code>test_fluidita</code> — 91 sezioni attraversate , cambio mediano 14 ms, peggiore 162 ms, 0 sezioni oltre 100 ms ; e le prove contrarie: la tela WebGL compare comunque, le 296 figure esistono tutte poco dopo, la stampa non esce muta
UI-10	L'interfaccia deve poter cambiare lingua fra molte, dichiarando per ognuna quanto è tradotta, portando con sé il verso di scrittura	<code>bsi-lingue.js</code> con quattordici lingue , selezionatore con ricerca, <code>dir="rtl"</code> per l'arabo	<code>test_lingue</code> — 170 elementi di scheletro su 170 in ognuna delle tredici lingue , copertura calcolata applicando il dizionario e contando; verso di scrittura provato nei due versi; categorie della barra e segnaposto della ricerca compresi
UI-11	La molecola deve mostrare come sta insieme, e l'angolo lo deve scegliere chi guarda	<code>bsi-mol3d.js</code> : formazione dalla polvere con arrivo scaglionato, e selezione degli atomi con un clic	<code>test_mol3d</code> — i pixel della tela a 130 ms e a 2,3 s, l'ordine d'arrivo (scheletro prima degli idrogeni) misurato sull'avanzamento di ogni atomo, e la selezione di 1, 2, 3 e 4 atomi

5. Requisiti di sicurezza e riservatezza

ID	Requisito	Implementazione	Banco di verifica
SEC-01	Nessuna chiave API deve essere presente nel repository	chiavi solo in <code>localStorage</code> o nei segreti del Worker	<code>tools/verifica-sicurezza.js</code> — 295 file tracciati, 8 forme di credenziale
SEC-02	Nessun dato personale deve lasciare il dispositivo senza azione esplicita	architettura local-first, telemetria disattivata	<code>audit_rete</code> — verifica diretta : un valore spia seminato in 71 depositi dei dati utente, l'applicazione usata per 91 sezioni su 6 pagine, e URL, intestazioni e corpo di ogni richiesta ispezionati. Più <code>verifica-sicurezza</code> sui due meccanismi di uscita
SEC-03	Le password non devono comparire in chiaro nel sorgente	SHA-256 in <code>file_manager.html</code>	<code>tools/verifica-sicurezza.js</code>
SEC-07	Nessuna credenziale deve essere MAI entrata nel repository, nemmeno in un commit poi corretto	nessuna chiave è mai stata committata; le chiavi stanno in <code>localStorage</code> o nei segreti del Worker	<code>audit_storia</code> — 1 551 versioni distinte di file di testo su 390 commit , 9 schemi provati nei due versi; fallisce su un clone superficiale, perché misurerebbe meno superficie
SEC-04	Nessun marcatore di conflitto deve raggiungere la pubblicazione	—	<code>verifica_guida</code> §12 — 58 file di testo

SEC-03 — limite dichiarato. GitHub Pages serve file statici: qualunque controllo di accesso lato pagina è aggirabile da chi legge il sorgente. La protezione del File Manager è un deterrente, **non** un controllo di sicurezza, e la documentazione lo afferma. Inoltre la password in chiaro è rimasta nella cronologia git fino alla sua rimozione, e togliere un segreto dai file non lo toglie dalla storia: `git log -p` lo restituisce a chiunque. L'unico rimedio effettivo era cambiarla, ed **è stato fatto** alla versione `bsi-v186`. Quella vecchia resta nella cronologia e non apre più niente.

6. Requisiti non ancora verificati automaticamente

Dichiarati per completezza. Sono coperti da ispezione documentale o da prova manuale, e la loro automazione è in programma.

ID	Requisito	Copertura attuale
UI-06	Conformità WCAG 2.1 AA completa	la parte meccanica è automatizzata su 13 pagine e 91 sezioni (tools/verifica-accessibilita.js , 33 642 elementi di testo: erano 19 751 prima che il testo su gradiente entrasse nella misura). Dei 1 069 difetti di contrasto emersi non ne resta nessuno: 0 misurati sulle stesse 91 sezioni, e il valore è registrato come riferimento che il banco difende. I campi senza etichetta sono zero : vedi il riquadro in docs/09 §4. Restano fuori il testo dentro gli SVG e quello su una vera immagine di sfondo, contati a ogni esecuzione (D-09)
PERF-01	Tempo di primo disegno su dispositivo di fascia bassa	prova manuale cross-device (docs/02 §4)
SCI-11	Strutture di 2 voci che non sono molecole singole (erano 6)	non rappresentabili : Ivermectina è una miscela di omologhi, Coartem un'associazione di due principi attivi. Le altre quattro sono state chiuse riprendendo la struttura da ChEMBL, vedi docs/06 §2.4
PERF-02	Copertura di codice dei banchi di prova	misurata parzialmente : 49,89 % di istruzioni sul percorso più ampio (audit_copertura). Resta non misurata la copertura dell'intera batteria e quella di rami; vedi §8
UI-07	Comportamento su Firefox e WebKit	non verificato — la batteria gira solo su Chromium; vedi §8

7. Copertura complessiva

Categoria	Requisiti	Verificati da banco	Copertura
Scientifici (SCI-01...23)	22	22	100 %
Agente AI (AI-01...10)	10	10	100 %
Stabilità (STA-01...08)	8	8	100 %
Interfaccia (UI-01...11)	9	9	100 %
Sicurezza (SEC-01...07)	5	5	100 %
Totale automatizzato	54	54	100 %
Non automatizzati (§6)	5	0	0 %
Totale dichiarato	59	54	91 %

La copertura è calcolata sui requisiti **dichiarati in questo documento** e non va confusa con una copertura di codice: misura quanti requisiti hanno un banco che li verifica, non quante righe vengono eseguite durante i test.

Perché due totali. Riportare solo il 100 % dei requisiti automatizzati sarebbe vero e fuorviante insieme: è il 100 % di ciò che si è scelto di automatizzare. Il numero che conta per chi valuta è il secondo — **54 requisiti verificati su 59 dichiarati** — e i cinque che mancano sono elencati per nome in §6, non riassunti in una percentuale.

Questa tabella è controllata da `tools/verifica-documenti.js`, che conta gli identificativi realmente presenti nel documento e fallisce se i totali qui scritti non corrispondono. È un controllo nato da un errore reale: i totali dicevano 36 su 9 categorie scientifiche mentre gli identificativi definiti erano 10, e SCI-10 compariva due volte — una come requisito verificato e una come requisito non verificato.

8. Lacune di copertura dichiarate

Non sono difetti: sono cose che **non sono state misurate**, e che quindi non possono essere affermate.

Lacuna	Che cosa comporta	Perché è così
Copertura di codice parziale	Misurata: 49,89 % di istruzioni sul percorso più ampio che un banco compie. Non è la copertura dell'intera batteria, ed è di istruzioni, non di rami	Il profilatore di Chromium la raccoglie nel motore, senza build e senza riscrivere il sorgente: l'ostacolo era dello strumento (c8, istambul), non del problema. Valore registrato, <code>audit_copertura</code> fallisce se scende
Solo Chromium	Il comportamento su Firefox e WebKit è verificato a mano, non da banco	La batteria usa <code>playwright-core</code> , che scarica un motore solo. Nell'ambiente di verifica la CDN degli altri motori risponde 403 alla politica di rete: non sono installabili lì
Nessuna regressione visiva	Un cambiamento grafico involontario non verrebbe intercettato	Gli spettri sono deterministici e confrontabili byte a byte (SCI-05): il confronto esiste sulle tracce, non sull'intera pagina
2 voci senza struttura	Due voci su 178 non hanno una struttura da mostrare: Ivermectina è una miscela di omologhi, Coartem un'associazione di due principi attivi. Erano sei	Vedi <code>06-Scientific-Accuracy-Data-Provenance.md</code> §2.4
Prestazioni su dispositivi lenti	Il tempo di primo disegno non è misurato su hardware di fascia bassa	Richiede dispositivi fisici; la prova è manuale

Documento aggiornato alla versione `bsi-v186`.